

IT3280 – THỰC HÀNH KIẾN TRÚC MÁY TÍNH

BÁO CÁO THỰC HÀNH

Tuần 10: Giao tiếp với các thiết bị ngoại vi

Họ và tên: Nguyễn Hồng Khôi

MSSV: 202416956

Assignment 1

Chương trình:

```
.eqv SEVENSEG_LEFT 0xFFFF0011 # Địa chỉ của đèn led 7 đoạn trái  
.eqv SEVENSEG_RIGHT 0xFFFF0010 # Địa chỉ của đèn led 7 đoạn phải
```

```
.text
```

```
main:
```

```
    li a0, 0x6D          # Mã Hex hiển thị số 5  
    jal SHOW_7SEG_LEFT    # Show số 5 bên trái
```

```
    li a0, 0x7D          # Mã Hex hiển thị số 6  
    jal SHOW_7SEG_RIGHT    # Show số 6 bên phải
```

```
exit:
```

```
    li a7, 10  
    ecall
```

```
end_main:
```

```
# -----
```

```
# Function SHOW_7SEG_LEFT : turn on/off the 7seg
```

```
# param[in] a0 value to shown
```

```
# remark t0 changed
```

```
# -----
```

```
SHOW_7SEG_LEFT:
```

```
    li t0, SEVENSEG_LEFT    # assign port's address
```

```

sb  a0, 0(t0)      # assign new value
jr  ra

# -----
# Function SHOW_7SEG_RIGHT : turn on/off the 7seg
# param[in] a0 value to shown
# remark t0 changed
# -----

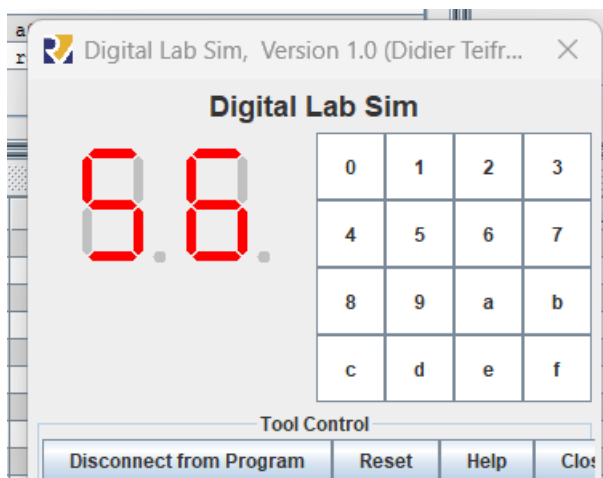
SHOW_7SEG_RIGHT:
    li  t0, SEVENSEG_RIGHT  # assign port's address
    sb  a0, 0(t0)          # assign new value
    jr  ra

```

Mô tả

Chương trình yêu cầu in ra 2 số cuối mssv, trong trường hợp này là 202416956 tức 56. Đèn trái được gán a0 = 0x6D kết quả sẽ là số 2, đèn phải Đèn trái được gán a0 = 0x7D kết quả sẽ là số 0.

Kết quả:



+ **LED trái (Số 5 - 0x6D):** Trạng thái các bit là 0110 1101. Các thanh sáng bao gồm a, c, d, f, g tạo thành hình số 5.

+ **LED phải (Số 6 - 0x7D):** Trạng thái các bit là 0111 1101. Các thanh sáng bao gồm *a, c, d, e, f, g* tạo thành hình số 6.

Assignment 2:

Chương trình:

```
.eqv SEVENSEG_LEFT 0xFFFF0011 # Địa chỉ của đèn led 7 đoạn trái
```

```
.eqv SEVENSEG_RIGHT 0xFFFF0010 # Địa chỉ của đèn led 7 đoạn phải
```

```
.data:
```

```
mask: .word 0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F
```

```
.text
```

```
main:
```

```
li a7,12
```

```
ecall
```

```
li t0, 10
```

```
div t1, a0, t0 # Lấy a0/10
```

```
rem t1, t1, t0 # Lấy chữ số hàng chục của a0
```

```
jal LED
```

```
jal SHOW_7SEG_LEFT # show
```

```
rem t1, a0, t0
```

```
jal LED
```

jal SHOW_7SEG_RIGHT # Lấy chữ số hàng đơn vị của a0

exit:

li a7, 10

ecall

end_main:

LED:

la t2,mask

slli t1,t1,2

add t1,t1,t2

lw a1,0(t1)

jr ra

SHOW_7SEG_LEFT:

li s0, SEVENSEG_LEFT

sb a1, 0(s0)

jr ra

SHOW_7SEG_RIGHT:

li s0, SEVENSEG_RIGHT # assign port's address

sb a1, 0(s0) # assign new value

jr ra

Mô tả

Chương trình yêu cầu hiển thị trên LED 7 đoạn 2 chữ số cuối của mã ASCII (ở hệ cơ số 10) của ký tự được nhập từ bàn phím.

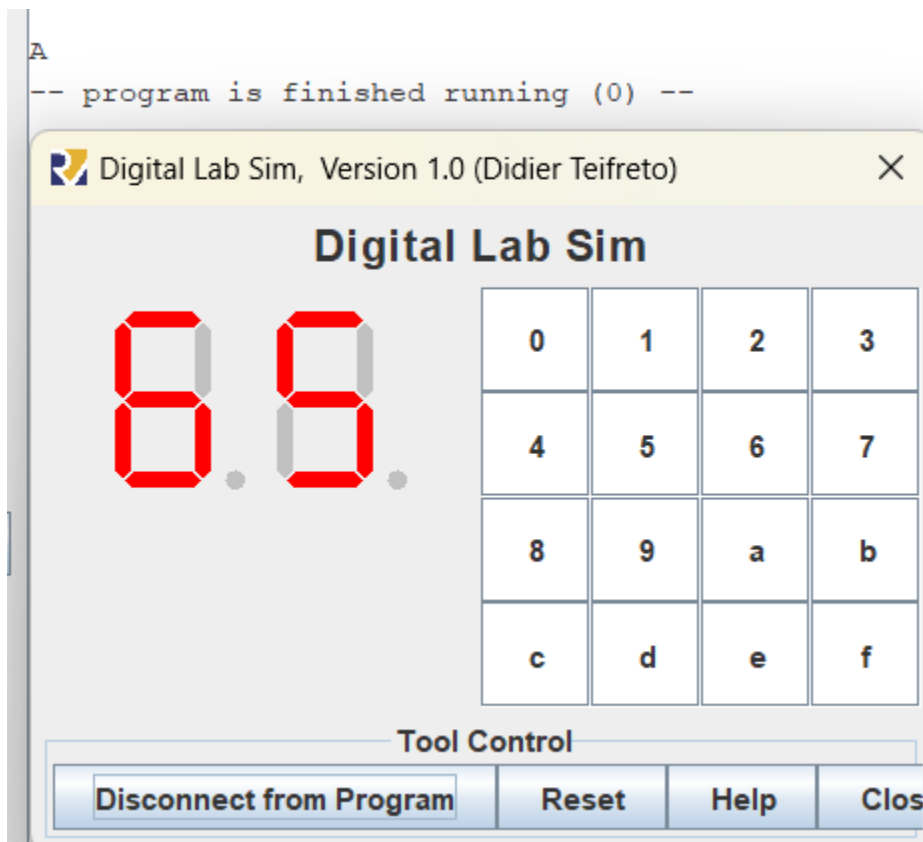
Khởi tạo 1 mảng LED gồm các số từ 1-9

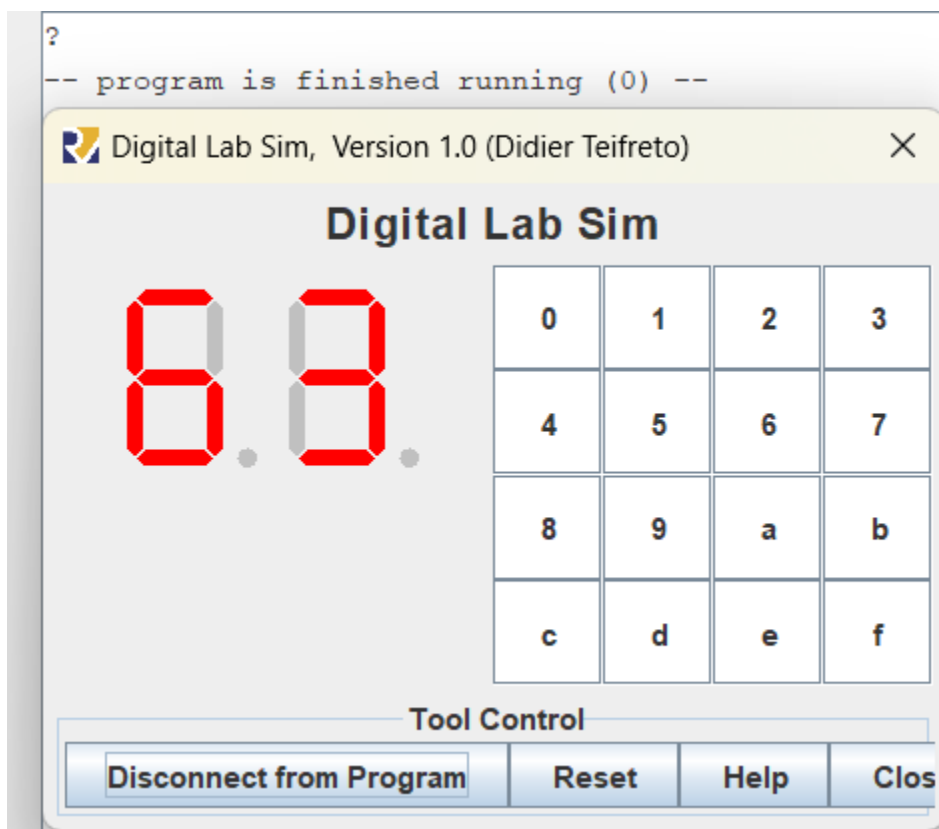
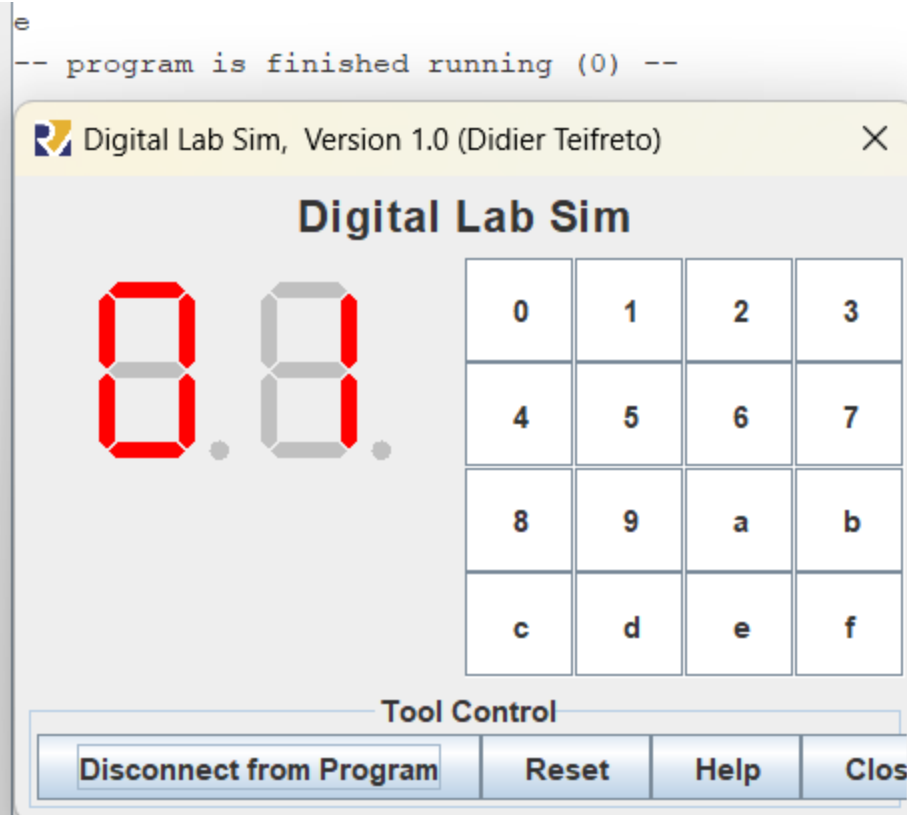
Lần lượt dùng lệnh div để lấy phần nguyên khi chia cho 10, lệnh rem để lấy phần dư khi chia cho 10 (chữ số hàng đơn vị).

Sau khi lấy được phần nguyên khi chia cho 10 thì tiếp tục dùng lệnh rem để lấy được chữ số hàng chục

Quy chiếu chữ số hàng chục và chữ số hàng đơn vị đến mảng và in ra như bài assignment 1.

Kết quả:





Assignment 3:

```
.eqv MONITOR_SCREEN 0x10010000
.eqv WARM_BROWN    0x00DAA06D # Màu nâu sáng
.eqv BROWN         0x008B4513 # Màu nâu đậm
.text
main:
    li a0, MONITOR_SCREEN    # a0 lưu địa chỉ gốc
    li a1, 0                  # a1 = 0: chỉ số hàng bắt đầu từ 0
    li t6, 8                  # t6 = 8

loop_hang:
    beq a1, t6, exit          # Nếu a1 == 8 thì đã vẽ xong tất cả hàng → thoát
    li a2, 0                  # a2 = 0: bắt đầu từ cột đầu tiên

loop_cot:
    beq a2, t6, next_row      # Nếu a2 == 8 thì đã vẽ xong hàng → sang hàng tiếp theo

    # Tính địa chỉ theo công thức: (row * 8 + col) * 4
    li t0, 8                  # t0 = 8 (số cột mỗi hàng)
    mul t1, a1, t0            # t1 = a1 * 8
    add t1, t1, a2            # t1 = t1 + a2 (chỉ số ô hiện tại)
    slli t1, t1, 2            # t1 = t1 * 4

    add t2, a0, t1            # t2 = địa chỉ ô cần tô màu
```



```
# Tính  $(a1 + a2) \% 2$  để xen kẽ màu
```

```
add t3, a1, a2
```

```
andi t3, t3, 1          #  $t3 = (a1 + a2) \& 1$  (nếu chẵn: 0, lẻ: 1)
```

```
beqz t3, is_brown       # Nếu  $t3 == 0$  (chẵn), tô màu nâu đậm (BROWN)
```

```
li t4, WARM_BROWN      # Ngược lại, tô màu nâu sáng
```

```
sw t4, 0(t2)
```

```
j next_col             # Chuyển sang ô tiếp theo
```

```
is_brown:
```

```
li t4, BROWN
```

```
sw t4, 0(t2)
```

```
next_col:
```

```
addi a2, a2, 1          # Tăng 1 cột
```

```
j loop_cot
```

```
next_row:
```

```
addi a1, a1, 1          # Tăng 1 hàng
```

```
j loop_hang
```

```
exit:
```

```
li a7, 10
```

```
ecall
```

Mô tả

Địa chỉ gốc là địa chỉ Monitor screen

Địa chỉ từng ô sẽ tính theo công thức: $(\text{row} * 8 + \text{col}) * 4$

Trong đó row là hàng (a1), col là cột(a2)

Sau đó đi tính $(a1+a2)\%2$ để xác định các ô chẵn và lẻ

Tô màu nâu đậm vào ô chẵn, tô màu nâu nhạt vào ô lẻ

Kết quả:



+ **Cấu trúc lưới:** Nhờ hai vòng lặp lồng nhau (duyệt hàng a1 từ 0–7 và duyệt cột a2 từ 0–7), chương trình đã xác định chính xác vị trí của 64 ô vuông trên màn hình.

+ **Màu sắc xen kẽ:** Thuật toán sử dụng phép toán $(\text{hàng} + \text{cột}) \% 2$.

- Các ô có tổng chỉ số chẵn được tô màu Nâu đậm (BROWN - 0x8B4513).
- Các ô có tổng chỉ số lẻ được tô màu Nâu sáng (WARM_BROWN - 0xDAA06D).

+ **Kết quả quan sát:** Trên cửa sổ Bitmap Display xuất hiện một hình ảnh dạng bàn cờ (checkerboard) kích thước 8x8 hoàn chỉnh, các ô màu sắp xếp đối xứng và không bị chồng lấn hay sai lệch địa chỉ.

Assignment 4:

```
.eqv KEY_CODE    0xFFFF0004
.eqv KEY_READY   0xFFFF0000
.eqv DISPLAY_CODE 0xFFFF000C
.eqv DISPLAY_READY 0xFFFF0008
```

```
.data
```

```
buffer: .space 4          # Bộ nhớ đệm 4 byte lưu 4 ký tự gần nhất để kiểm tra
                           chuỗi "exit"
```

```
.text
```

```
main:
```

```
    li a0, KEY_CODE        # địa chỉ đọc ký tự từ bàn phím
    li a1, KEY_READY        # địa chỉ cờ sẵn sàng bàn phím
    li s0, DISPLAY_CODE     # địa chỉ ghi ký tự ra màn hình
    li s1, DISPLAY_READY    # địa chỉ cờ sẵn sàng màn hình
    la s2, buffer           # địa chỉ mảng buffer dùng lưu 4 ký tự cuối
```

```
loop:
```

```
WaitForKey:
```

```
    lw t1, 0(a1)           # Đọc KEY_READY vào t1
```

```
beq t1, zero, WaitForKey
```

```
lw t0, 0(a0) # t0 ← KEY_CODE
```

```
# Cập nhật buffer: đẩy 3 ký tự cũ về trước 1 vị trí
```

```
lb t2, 1(s2) # Đọc buffer[1] vào t2
```

```
sb t2, 0(s2) # Ghi vào buffer[0]
```

```
lb t2, 2(s2) # Đọc buffer[2]
```

```
sb t2, 1(s2) # Ghi vào buffer[1]
```

```
lb t2, 3(s2) # Đọc buffer[3]
```

```
sb t2, 2(s2) # Ghi vào buffer[2]
```

```
sb t0, 3(s2) # Ghi ký tự mới vào buffer[3]
```

```
# Kiểm tra xem 4 ký tự cuối có phải "exit" không
```

```
lb t2, 0(s2) # Đọc buffer[0]
```

```
li t3, 'e' # t3 ← mã ASCII 'e'
```

```
bne t2, t3, SkipExit # Nếu buffer[0] khác 'e' thì bỏ qua
```

```
lb t2, 1(s2) # Đọc buffer[1]
```

```
li t3, 'x' # t3 ← mã ASCII 'x'
```

```
bne t2, t3, SkipExit # Nếu buffer[1] khác 'x' thì bỏ qua
```

```
lb t2, 2(s2) # Đọc buffer[2]
```

```
li t3, 'i' # t3 ← mã ASCII 'i'
```

```
bne t2, t3, SkipExit      # Nếu buffer[2] khác 'i' thì bỏ qua
```

```
lb t2, 3(s2)             # Đọc buffer[3]
```

```
li t3, 't'               # t3 ← mã ASCII 't'
```

```
bne t2, t3, SkipExit      # Nếu buffer[3] khác 't' thì bỏ qua
```

```
# Nếu cả 4 ký tự là "exit" → thoát chương trình
```

```
li a7, 10                # syscall code 10 = exit
```

```
ecall                    # Gọi hệ thống để kết thúc chương trình
```

SkipExit:

WaitForDis:

```
lw t1, 0(s1)             # Đọc DISPLAY_READY vào t1
```

```
beq t1, zero, WaitForDis
```

Encrypt:

```
# Kiểm tra nếu ký tự là chữ số (0–9): giữ nguyên
```

```
li t2, '0'               # t2 = '0'
```

```
blt t0, t2, NotNumber     # Nếu t0 < '0' thì không phải số
```

```
li t2, '9'               # t2 = '9'
```

```
ble t0, t2, ShowKey        # Nếu '0' <= t0 <= '9' thì giữ nguyên ký tự
```

NotNumber:

```
# Nếu là chữ hoa (A–Z): chuyển sang chữ thường (a–z)
```

```
li t2, 'A'               # t2 = 'A'
```

```
blt t0, t2, NotUpper    # Nếu t0 < 'A' thì không phải chữ hoa
li t2, 'Z'              # t2 = 'Z'
ble t0, t2, ToLower     # Nếu 'A' <= t0 <= 'Z' thì chuyển thường
```

NotUpper:

```
# --- Nếu là chữ thường (a-z): chuyển sang chữ hoa (A-Z) ---
li t2, 'a'              # t2 = 'a'
blt t0, t2, ToStar      # Nếu t0 < 'a' thì không phải chữ thường
li t2, 'z'              # t2 = 'z'
ble t0, t2, ToUpper     # Nếu 'a' <= t0 <= 'z' thì chuyển hoa
```

ToLower:

```
addi t0, t0, 32         # chuyển chữ Hoa thành chữ thường
j ShowKey
```

ToUpper:

```
addi t0, t0, -32        # chuyển chữ thường thành chữ Hoa
j ShowKey
```

ToStar:

```
li t0, '*'              # Các ký tự còn lại chuyển thành '*'
```

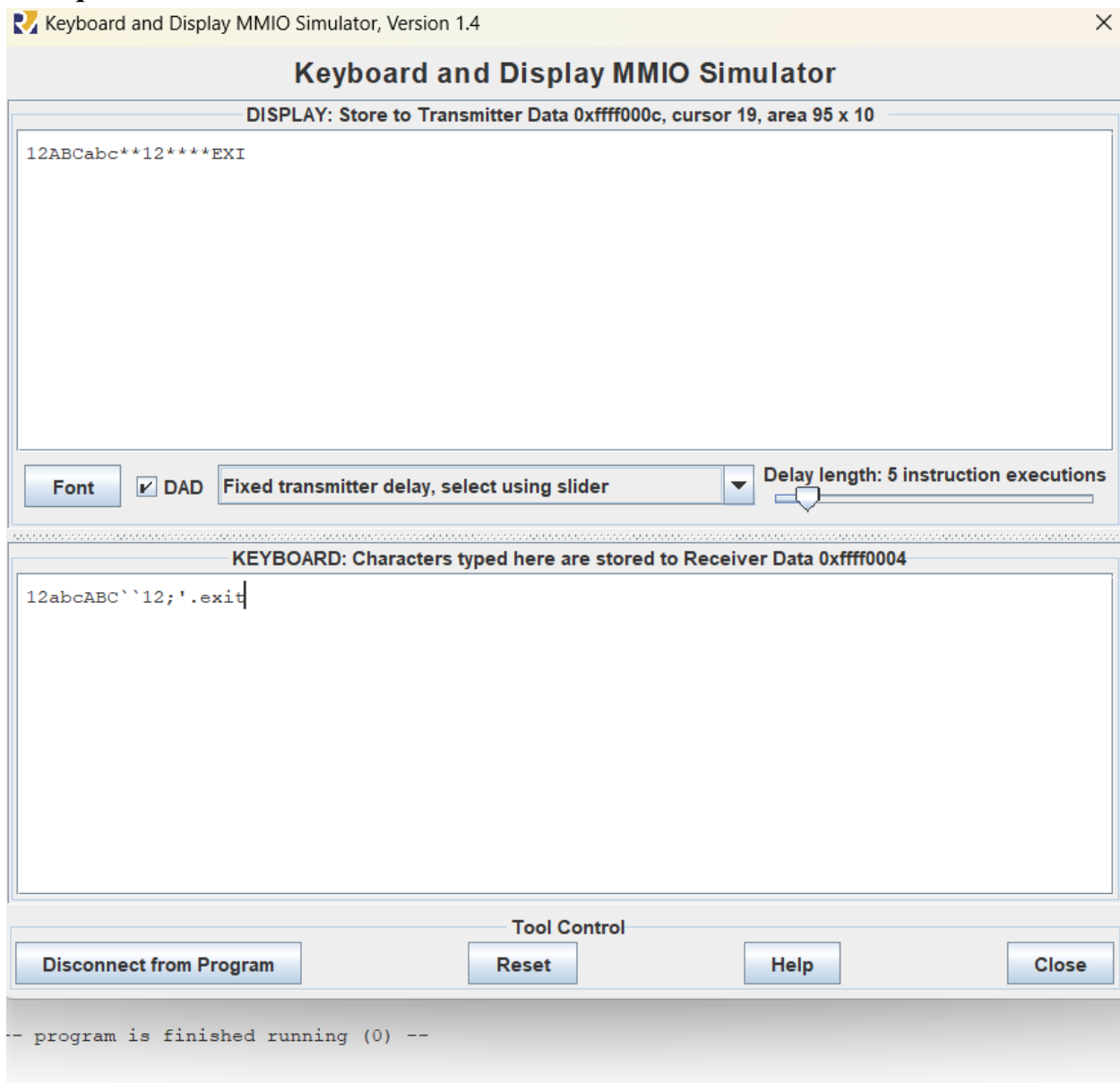
ShowKey:

```
sw t0, 0(s0)
j loop
```

Mô tả:

Dùng 1 stack buffer để check 4 kí tự nhập gần nhất có trùng với chuỗi exit không, lần lượt check buffer[0] với e, buffer[1] với x, buffer[2] với i, buffer[3] với t, nếu giống hoàn toàn thì thoát chương trình. Nhập 1 kí tự mới sẽ đẩy 1 ký tự cũ ra để tạo thành 1 chuỗi 4 ký tự mới và lại check với exit.

Kết quả:



=> Chương trình đã thực hiện thành công việc tương tác với thiết bị ngoại vi

thông qua sơ đồ ánh xạ bộ nhớ (MMIO). Kết quả thực nghiệm cho thấy khả năng xử lý dữ liệu thời gian thực: khi nhập 'A' kết quả hiển thị là 'a', nhập 'b' hiển thị 'B', và nhập các ký tự đặc biệt sẽ hiển thị '*'. Đặc biệt, cơ chế nhận diện chuỗi điều khiển 'exit' hoạt động chính xác, cho phép người dùng thoát trình giả lập một cách an toàn mà không cần can thiệp vào trình điều khiển hệ thống.